AI Speech Coach: A Real-Time Multimodal System for Analyzing Speech Delivery and Voice Emotion.

A **B.TECH PROJECT-II** REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR
THE AWARD OF THE DEGREE
OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER ENGINEERING

Submitted By

Abhinaba Bhakat	**Aman Meena**	**Manish Singh Rawat**
(2K22/CO/13)	**(2K22/CO/51)**	**(2K22/CO/270)**

Under the supervision of
Prof. Vinod Kumar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DELHI TECHNOLOGICAL UNIVERSITY
(Formerly, Delhi College of Engineering)
Bawana Road, Delhi-110042

May 2026

CANDIDATE'S DECLARATION

We, Abhinaba Bhakat (2K22/CO/13), Manish Singh Rawat (2K22/CO/270), Aman Meena (2K22/CO/51), pursuing Bachelor of Technology degree in Computer Engineering from the Department of Computer Science and Engineering, hereby declare that mini project report titled "AI Speech Coach: A Real-Time Multimodal System for Analyzing Speech Delivery and Voice Emotion" submitted by us to the Department of Computer Science and Engineering, Delhi Technological University, Delhi, in partial fulfillment for the degree, is completely original and not copied from any other source without proper citation. We declare that this is our own work and that it has not been submitted in part or in whole for any degree or diploma to this or any other University.

(Signature)
Abhinaba Bhakat (2K22/CO/13)

(Signature)
Manish Singh Rawat (2K22/CO/270)

(Signature)
Aman Meena (2K22/CO/51)

Place : New Delhi
Date: 25/05/2026

CERTIFICATE

This is to certify that the work entitled "AI Speech: A Real-Time Multimodal System for Analyzing Speech Delivery and Voice Emotion" submitted by Abhinaba Bhakat (2K22/CO/13), Manish Singh Rawat (2K22/CO/270), Aman Meena (2K22/CO/51), of Department of Computer Science and Engineering, Delhi Technological University, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Delhi Technological University, has been carried out by the students under my supervision. I am not aware that this work or any part thereof has been submitted for any degree or diploma to this University or to any other University.

(Signature)
Prof. Vinod Kumar

Designation of Supervisor

Place : New Delhi

Date : 25/05/2026

ACKNOWLEDGEMENT

We are immensely grateful to our supervisor Vinod Kumar, Professor of Department of Computer Science and Engineering, Delhi Technological University. They gave us their time for free, asked the tough questions at the right times and kept pushing the work toward something we could be proud of. We got stuck sometimes, but a quick chat with them usually got us unstuck. We are also thankful to the Head of the Department and to the faculty of the Department of Computer Science and Engineering, who created the kind of environment, and provided the lab resources, which allowed a project like this to happen. And thanks to the technical and administrative staff as well for their quiet, steady help during the semester. Finally, to our families and friends: thank you for the patience and the encouragement when the late nights stacked up. We couldn't do it without you.

(Signature)
Abhinaba Bhakat (2K22/CO/13)

(Signature)
Manish Singh Rawat (2K22/CO/270)

(Signature)
Aman Meena (2K22/CO/51)

ABSTRACT

Everyone's a little scared of speaking in front of others and no one gets honest feedback while they practice. Coaches are expensive and have to be booked in advance. Rather, the apps people reach for tend to either simply record them or give back a word count, which tells you nothing about whether they really sounded convincing. That's why we built the AI Speech Coach, it listens as you speak, shows you what is happening when it is happening and then writes you a short coaching report. The system consists of two halves. A Next.js web app that captures the microphone and streams the audio in small PCM frames over a WebSocket to a Python backend running FastAPI. On the server, a library pipeline tracks the things a speaker can actually change in the moment, namely pitch, volume, energy and pace, while a separate engine we call ConvinceSense judges tone and persuasiveness. ConvinceSense operates on two clocks. The emotion is read from the speaker's voice in eight classes every two seconds or so, using a Wav2Vec2-XLSR model fine-tuned on RAVDESS. Every three seconds, a second stage takes the words and runs a DistilBERT sentiment model and a DistilRoBERTa emotion model on the transcript that Deepgram Nova-3 produces live. Then a small Random-Forest model folds the acoustic and linguistic features into one convincingness score from one to five. Nothing from the raw audio is ever written to disk. Only the derived signals, the transcript, the metrics and the emotion events are stored in a small SQLite database. When the speaker is ready, a locally running Llama 3.2 model via Ollama turns all of that into a warm, specific coaching report saved as a PDF and a JSON file. The complete stack for analysis runs on the user's own machine, keeping it private and keeping the cost at zero per session. This report describes why we built the system, what preceded it, what we set out to do, how the pieces fit together, what we found when we ran it, and where we think it should go next.

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbrevations/ Symbols	Description
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
SER	Speech Emotion Recognition
ASR	Automatic Speech Recognition
STT	Speech-to-Text
NLP	Natural Language Processing
LLM	Large Language Model
API	Application Programming Interface
WS	WebSocket
REST	Representational State Transfer
MFCC	Mel-Frequency Cepstral Coefficients
RMS	Root Mean Square (energy)
ZCR	Zero-Crossing Rate
F0	Fundamental Frequency (Pitch)
WPM	Words Per Minute
PCM	Pulse-Code Modulation
XLSR	Cross-Lingual Speech Representations
SST-2	Stanford Sentiment Treebank (2-class)
RF	Random Forest
SSL	Self-Supervised Learning
VAD	Voice Activity Detection
EMA	Exponential Moving Average
DSP	Digital Signal Processing
CORS	Cross-Origin Resource Sharing
DB	Database
PDF	Portable Document Format
JSON	JavaScript Object Notation
UI	User Interface
CPU	Central Processing Unit
GPU	Graphics Processing Unit

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background

Speaking well in front of other people is one of those skills that quietly decide a lot. A seminar, an interview, a sales pitch, a demo day, a lecture, the Monday stand-up: the delivery of an idea is almost as important as the idea itself. Steady, expressive, confident delivery earns trust, commands a room. A rushed, flat, filler-heavy one undermines even good stuff. And yet, most surveys say the fear of public speaking stubbornly sits near the top of what people fear, ahead of plenty of things that ought to frighten us more.

To improve the normal way is to practice with someone who knows what to look for. A coach can sit with you, notice that you speed up when you are nervous or trail off at the end of sentences and give you something concrete to work on. The problem is that this kind of help is costly, has to be booked in advance, and is just not an option for most students or people at the beginning of their careers. So most of us practice alone, without really having any sense of how we sound to anyone else. We might record ourselves and play it back, but a raw recording doesn't analyze anything. It won't tell you that you said 'basically' eleven times, that your pitch never moved, or that you sped up the moment the topic got hard.

What has changed in recent years is that the machine-learning tools needed to do that analysis have become good enough and cheap enough to run on an ordinary laptop. Models that can recognize speech, read emotion from a voice, classify the mood of a sentence and write fluent feedback are no longer research curiosities. That is the opening we wanted to exploit: take these pieces, which normally live in separate papers and repositories, and wire them into one real-time coach that anyone can run privately, as many times as they want, for free.

1.2 Why Delivery Matters

Employers, when asked what they want from graduates, consistently put communication skill at the top of the list, often above narrow technical ability. It's no mystery. You have to justify and defend technical work before it actually counts for something. The designer who can defend a design. The researcher who can defend a thesis. The founder who can make an investor lean in. They're all leaning on the same underlying skill.

But delivery isn't one thing and that's good news because it means it can be measured. Pace (words per minute) determines whether the audience can follow. What separates a live speaker from a sleeping one is pitch variation. Consistent volume reads as confidence and, more

practically, keeps you audible. Like, um, uh, filler words leak nervousness and chip away at authority. And the emotional tone of a voice, whether it sounds calm, eager or anxious, affects how the room responds to both the speaker and the message. All of these can be picked up automatically, and this is exactly what we wanted to do.

There is also a quieter argument for an automated coach, and that is repetition. You get better at anything by doing it over and over and getting feedback, but a human coach can only realistically sit in on a few of your practice runs. Software never gets tired. It will follow your tenth rehearsal as closely as it does your first, at midnight if that's when you practise, and it can keep score across all of them so that you can see whether you are actually improving. It is that mix of objectivity, patience and availability that makes this worth building as a companion to human coaching, not a cut-rate replacement for it.

1.3 Where Current Tools Fall Short

See what can be found today and a pattern emerges. Human coaching is effective, but it doesn't scale and it isn't cheap. Plain recording apps record audio, that's all. Some products do report metrics, but they tend to stay at the surface, total talking time and a word count, and many of them ship your voice off to a server somewhere, which raises both a privacy question and a recurring bill. There are very few tools that take your sound and what you say and turn it into a single, readable verdict on how convincing you were. Fewer still take their numbers and write back the warm, prioritized advice a real coach would give.

That's the hole we were after. We wanted something that is live, looks at both the voice and the words, keeps everything on the user's own machine, and still manages to produce feedback that reads like a person wrote it. The next few chapters explain how we arrived at this point.

1.4 Problem Statement

The problem, in a nutshell, is this: develop a system that provides a speaker with honest, unbiased, emotion-aware feedback while they're actually speaking, and then converts what it hears into useful coaching, all without the use of a paid external service and without ever storing the raw voice of the person. All those clauses cost money. It means streaming audio from a browser to a server with little delay, pulling reliable features out of that audio on the fly, transcribing speech as it arrives, judging emotion and persuasiveness from both the sound and the transcript and finally turning a running stream of numbers into advice a nervous student would find encouraging rather than clinical.

1.5 Our Approach

The answer is the AI Speech Coach. The user is building a web app in Next.js. It accesses the microphone and streams 16-bit PCM frames over a WebSocket to a FastAPI backend. The backend does double work. Computes live delivery metrics with librosa. Runs ConvinceSense, the engine that combines a voice-emotion model with a transcript reader to generate a rolling convincingness score. Deepgram Nova-3 transcribes it live. When the speaker is done and asks for a report, a Llama 3.2 model running locally through Ollama generates the coaching narrative that we then export as PDF and JSON. The entire analysis runs locally, and only the derived signals are saved, never the audio itself, which is what keeps the whole thing private and essentially free to run.

1.6 Contributions

The major contributions of this project are:

- An end-to-end, real-time speech coach that works, that covers a web client, an asynchronous Python backend, and models that are hosted locally, that you can actually sit down and use.
- ConvinceSense, a multimodal engine that runs a deep voice-emotion model and a transcript reader on two different clocks and combines them into one convincingness score from one to five.
- An example of a locally hosted language model writing genuinely useful coaching, with no paid API in the loop.
- A design that never stores raw audio, only metrics and transcripts, which addresses the privacy concern that voice apps usually stumble over.
- A write-up of an ugly, under-documented trap in deploying self-supervised emotion checkpoints, and how we worked around it.

1.7 Scope

This is a one-user proof of concept and we've tried to be upfront about that. It has microphone input only, live dashboard for pace, fillers, volume, pitch and emotion, multimodal emotion and convincingness analysis, report generation on demand and persistence of the derived data. We intentionally left out video and body language, multi-speaker or audience detection, login and multi-user accounts, and a native mobile app. Those cuts kept the work focused, and we revisit most of them in Chapter 6 as future work.

1.8 How This Report Is Laid Out

Chapter 2 reviews the previous work we built upon, including speech emotion recognition, text emotion analysis, speech recognition, feature extraction and fusion, and language models for feedback, and ends with an evaluation of existing coaching tools. In chapter 3, we explain the gaps we found, the objectives we derived from the gaps and our requirements. Chapter 4 is the long one: the full design, all the major components, the data model, the schema and the stack. In Chapter 5, we present what we built and what we observed. Chapter 6 concludes and looks ahead. Chapter 7 contains the timeline and the references and list of publication conclude the report.

2. LITERATURE SURVEY

2.1 Overview

The AI Speech Coach is not the result of a single idea but of many mature ideas woven together. In this chapter, we discuss previous work that was relevant to us: emotion recognition from speech, emotion recognition from text, speech recognition, extraction of acoustic features from audio, fusion of multiple signals and leveraging large language models for feedback generation. Finally, we present current coaching tools and position our contribution within this landscape.

2.2 Recognising Emotion from Speech

Speech Emotion Recognition, often abbreviated to SER, is the task of predicting a person's emotion from the sound of their voice alone, from factors such as pitch variation, the form of the energy over time, rhythm and spectral colour. It is an age-old problem in affective computing, the methods having entered roughly three eras.

The first era used hand-crafted features, designed by researchers, pitch statistics, energy, speaking rate, and Mel-Frequency Cepstral Coefficients, fed to classifiers such as support vector machines or hidden Markov models. They were light weight and easy to reason about, but they only saw as far as their hand-built features allowed them to see and tended to break when the speaker, microphone or language changed.

The second era delegated feature design to neural networks, usually convolutional or recurrent, operating on spectrograms. Accuracy improved but these models are hungry for labelled data and labelled emotional speech is scarce and expensive to collect.

The third era, our era, is the era of self-supervised learning. You learn rich speech representations from huge piles of unlabelled audio with wav2vec 2.0 [1] and then fine-tune them for a downstream task (like SER) with comparatively little labelled data. Its cross-lingual sibling, XLSR [2], extends that pre-training to many languages, and the representations transfer surprisingly well even to languages it never saw during fine-tuning. Related encoders such as HuBERT [16] (learned by predicting masked hidden units) and WavLM [15] (adding a denoising step to handle noisy audio) have set the state-of-the-art on a wide range of speech benchmarks. emotion2vec [5] is an even newer idea, pre-training specifically for emotion rather than for general speech, and it has strong numbers among open SER models.

None of this happens without data sets. RAVDESS [3] is a clean dataset of acted emotional speech and song in North American English, recorded in a studio, and is popular specifically

because the recordings are tidy, and the classes are balanced. IEMOCAP [4] is a standard benchmark containing emotional conversations between two people, but it is acted and does not always transfer to spontaneous speech. For our acoustic channel we chose a Wav2Vec2-XLSR model fine-tuned on RAVDESS that produces eight classes: angry, calm, disgust, fearful, happy, neutral, sad and surprised. We chose it because it is accurate, it has eight-way granularity, and it runs on short windows on a CPU. The acoustic models that were compared are shown in Table 1.

Model	Backbone	Strengths	Why not (for us)
Wav2Vec2-XLSR (RAVDESS)	wav2vec 2.0 / XLSR	8 classes, accurate, CPU-friendly on 2 s windows	Trained on acted speech; head must be loaded carefully
emotion2vec	Emotion-specific SSL	Purpose-built for emotion	Heavier dependency footprint
WavLM-Large	WavLM	State of the art, noise-robust	Too heavy for real-time CPU work
HuBERT / DistilHuBERT	HuBERT	Strong; distilled version is quick	Lower SER accuracy than purpose-built models
SpeechBrain wav2vec2 (IEMOCAP)	wav2vec 2.0	Trivial to deploy	Only 4 classes; acted-speech domain gap

Table 1: Comparison of Acoustic Speech-Emotion-Recognition Models

2.3 Sentiment and Emotion from Text

Acoustic models listen to how something is spoken. Text models read what is said, and the two rarely agree by accident, which is why it's worth combining them. Transformer language models have become the de facto standard in the task of text classification. BERT introduced deep bidirectional pre-training; RoBERTa [9] tweaked the recipe to get better results; and DistilBERT [8] used distillation to create a model that is approximately forty percent smaller and sixty percent faster, while still retaining most of the accuracy, which is why it is usable in real time.

We use DistilBERT fine-tuned on SST-2 for sentiment. It makes a positive or negative call with a confidence. If it is not very confident about a negative label, we treat the result as neutral. This fills in the hole left by a two-class scheme. For finer emotion we use the DistilRoBERTa model by Hartmann [7], trained on GoEmotions [6], and reduced to seven classes: anger, disgust, fear, joy, neutral, sadness and surprise. It reads the emotional content of the words and balances the acoustic channel, which can be tricked by tone alone. At the bottom of all this is the self attention mechanism introduced by Vaswani et al. [18].

2.4 Speech Recognition

You cannot analyse words you have not transcribed and transcription during live speech has to be both accurate and fast. Two families do the work. Open models like Whisper [13] are powerful offline, robust to accents and noise thanks to large-scale weak supervision, and can run locally, making them a good safety net when the network or a paid service is not available. Streaming services such as Deepgram Nova-3 [19] are tuned for the live case, and will give you word-level timestamps, low-latency interim results, voice-activity events and, handy for a coach, explicit recognition of filler words. If a key is configured, we use Deepgram Nova-3, otherwise we fall back to a local Whisper model. This gives us the ability to trade off accuracy, latency and resilience, without rewriting anything.

We also had a free resilience story running both engines. Once the network is up and a key is configured, Deepgram returns the cleaner, lower-latency transcript with the filler words flagged. When it is not, the local Whisper model keeps the linguistic side alive offline, in line with the privacy-first spirit of the rest of the system. Both paths feed the same transcript handling code, so everything downstream sees the switch as invisible.

2.5 Acoustic Features

It's not all about learned representations. Classic signal processing features are still worth keeping, both for the live meters and as inputs to a light fusion model. librosa [10] is the workhorse here, with efficient code for MFCCs, root-mean-square energy, zero-crossing rate, and spectral contrast. Special mention to pitch, because it has so much to do with both delivery and emotion, and estimating it reliably is a classic headache. The YIN algorithm [11] is an autocorrelation-based estimator which works well for a single speaking voice, and is much more robust for speech than finding peaks in an FFT. The probabilistic version of YIN, pYIN, also tells you which frames are voiced. The live pitch track uses YIN and the fusion feature extractor uses pYIN.

2.6 Combining Modalities

When you combine the acoustic and the linguistic evidence, you generally get a more reliable reading than either provides alone, because the two channels are only partially correlated. You can fuse early, by gluing feature vectors together before classification. Late, by mixing the outputs of separate classifiers. Or in between, with a model trained to do the mixing. For a small, tabular dataset with multimodal features, a Random Forest [12] is a good option: it's fast to train and run, hard to overfit, reasonably interpretable via feature importances, and runs fine on a

CPU. We chose to use early fusion, concatenating the acoustic features with an encoded linguistic vector and sending the result to a Random Forest that outputs a convincingness score between one and five.

We did look at late fusion, where you have a classifier for each modality and then you average or vote on the outputs. Simpler, and nicely decouples the channels, but you lose the cross-modal interactions that are often the most telling, and you have to calibrate two separate confidence scales before you can combine them sensibly. Early fusion avoids both these problems, but at the cost of a single model to deal with a mixed-up feature vector, which is precisely the kind of thing a Random Forest can deal with without complaint.

2.7 Language Models for Feedback

We can frame turning a pile of metrics into advice that sounds human as a text-generation problem, and instruction-tuned large language models are good at that. We show that a model of the Llama family [14] can be prompted with the session metrics and a transcript in a structured way to generate a coherent and specific narrative that even refers back to the data and ranks its suggestions. The interesting part was the deployment option. Rather than use a paid cloud API, we run an open Llama 3.2 model locally using Ollama. That kills the per-request cost, removes the need for an API key, keeps the transcript on the user's machine and makes the system self-contained. The tradeoff is slower generation on a CPU.

Prompt is most of what gets good output. We give the model a system message that presents the model as an experienced, warm, specific speaking coach, and a user message with the real metrics and a piece of the transcript, and then ask it to respond in named sections and to end with three ranked, do-this-next improvements. The structure is pinned down, and therefore predictable enough to render, but the model has room to write something that fits the individual speaker. Limiting length and choosing a mediocre temperature prevents generation from going long or rambling on modest hardware.

2.8 Existing Coaching Tools

There are several commercial and academic products that address pieces of this. Most select a subset of the relevant signals. Many depend on the cloud. Table 2 juxtaposes what such tools typically do with our goals. The same recurring shortfalls were mentioned earlier, no real fusion into one readable measure, a dependence on remote servers with the privacy and cost baggage that brings, and feedback that is statistical and not written like advice.

Capability	Typical existing tools	AI Speech Coach (this work)
Real-time delivery metrics	Partial	Yes (pace, fillers, volume, pitch)
Acoustic emotion from voice	Rare	Yes (Wav2Vec2-XLSR, 8 classes)
Linguistic emotion / intent	Rare	Yes (DistilBERT + DistilRoBERTa)
Single fused persuasiveness score	No	Yes (Random Forest, 1-5)
Narrative AI coaching report	Limited	Yes (local Llama 3.2)
Runs fully locally / private	Usually cloud	Yes (no raw audio stored)

Table 2: Comparison of Existing Speaking-Coach Systems

2.9 Affect, Ethics and Evaluation

Affective computing, in general, is about systems that detect and respond to human emotion, and speech-based affect is one of its busiest corners. They have applications in call-centre analytics, mental-health monitoring, tutoring, and, as here, communication coaching. One idea repeated again and again in the literature is that emotion inference is inherently uncertain and speaker-dependent and so a responsible system would present it as a signal with a confidence attached, not as fact. That's why every emotion and convincingness number we show comes with a confidence, and why the wording the user sees frames things as observations, not verdicts.

When people evaluate these systems, they tend to report accuracy, weighted and unweighted recall, F1, and confusion matrices against held-out labels. The issue is the gap between acted corpora, which are clean and balanced but staged, and the messiness and lopsidedness of real, spontaneous speech. That gap is a big part of why a model trained on acted data, like the RAVDESS-fine-tuned checkpoint we use, does its best work on clean input and gets shakier as background noise creeps in, something we saw ourselves and report in Chapter 5.

2.10 Where This Work Fits

So all the building blocks are there and they are good: self-supervised acoustic emotion models, transformer text classifiers, low-latency streaming transcription, robust pitch estimation, ensemble fusion, language models you can host yourself. What we did not find was one single practical privacy respecting tool that pulls them into one real-time pipeline, fuses how you sound with what you say into one number and then writes that up as coaching a person would actually read. That's the space the AI Speech Coach plays in, and Chapter 3 lays out the specific gaps it fills in.

3. OBJECTIVES AND RESEARCH GAPS

With the previous work in mind, this chapter identifies the gaps that led us to build the system, states the objectives we derived from them, and writes down the requirements, functional and non-functional, that the system had to fulfill.

3.1 The Gaps We Saw

Reading around the field, there were five gaps that stood out and the project shaped around closing those gaps:

G1. No one readable measure of persuasion. Consumer tools give you shallow statistics and almost never fold how you sound together with what you say into one interpretable number a speaker can watch and chase.

G2. Over-reliance on the cloud. Most emotion-aware speaking aids route the voice through a remote API, which costs money over time and, more importantly, raises a real privacy concern. Missing a pipeline that runs locally but still writes good feedback.

G3. One window for all. Acoustic cues and linguistic cues are at different natural time scales but many systems jam them into one analysis window. A design that enables each modality to run at its own pace is barely explored in shipping tools.

G4. SER checkpoints misloaded. The off-the-shelf self-supervised emotion checkpoints are often misused. Loading a checkpoint with a custom head through a generic pipeline can silently discard the trained head and randomly re-initialise it, leading to nonsense. This trap hasn't been written about much.

G5. Guidance, not numbers. And even the tools that gather rich data tend to dump it as charts and figures, rather than the ranked, encouraging, specific advice a human coach would give.

3.2 Objectives

To close those gaps, we set the following objectives for ourselves:

O1. (G2) Capture audio from a microphone in the browser and stream it to a backend with low latency for continuous analysis.

O2. Show live delivery metrics, pace in words per minute, filler frequency, volume and pitch on a dashboard the speaker can see at a glance (comes after G5).

O3. Read emotion from the voice with a deep SER model on short windows. Load the trained classifier head correctly (goes after G3 and G4).

O4. Read transcript for sentiment, emotion and intent and fuse acoustic and linguistic evidence into one convincingness score (goes after G1 and G3).

O5. Generate an empathetic, data-driven coaching report with locally-hosted language model on demand, and export as PDF and JSON (follows G2 and G5).

O6. Persist session metadata and derived signals, but never raw audio (goes after G2).

3.3 Functional Requirements

The functional requirements describe the system's required behavior. They are placed in Table 8 with the non-functional ones.

- FR1. The user can create an analysis, name it, and log one or more sessions in that analysis.
- FR2. As you record, the system streams your audio back and provides live metrics, emotion, a convincingness score and transcript updates.
- FR3. The system detects filler words, counts them and the speaking rate.
- FR4. The system detects the voice emotion class and computes a convincingness score in real time.
- FR5. The system generates and stores a coaching report upon request and enables the user to download it.

3.4 Non-Functional Requirements

- NFR1. Privacy: raw audio is never written to disk, neither on the client nor the server.
- NFR2. Latency: live metrics update about four times a second, and never block on a slow model call.
- NFR3. Cost: The analysis and report stack is local and there is no required paid service.
- NFR4. Robustness: The system bends, but not breaks, in case the transcription service or the trained fusion model, an optional dependency, is missing.
- NFR5. Resource use: model loading is managed so that it fits into the memory of a normal machine.

Type	ID	Requirement
Functional	FR1	Create and name analyses; record multiple sessions
Functional	FR2	Stream audio and return live metrics/emotion/score/transcript
Functional	FR3	Detect filler words and compute pace
Functional	FR4	Classify voice emotion and convincingness in real time
Functional	FR5	Generate, store, and download a coaching report
Non-functional	NFR1	Never persist raw audio (privacy)
Non-functional	NFR2	Roughly quarter-second live-metric cadence (latency)

Non-functional	NFR3	Run locally without paid services (cost)
Non-functional	NFR4	Bend, don't break, on missing dependencies
Non-functional	NFR5	Bounded memory on commodity hardware

Table 8: Functional and Non-Functional Requirements

4. METHODOLOGY

This is the main chapter. It's a step-by-step explanation of how the AI Speech Coach is made. The architecture is a typical client-server architecture: a browser captures and transmits audio, and a python server does the signal processing, model inference, storage, and report writing.

4.1 System Overview

Here's the journey from the user's point of view. You create a named analysis, capture one or more sessions in that analysis while you watch a live dashboard, and when you feel ready, you ask for a coaching report. Behind that, the browser streams audio to the backend over a WebSocket. The backend performs the live metrics calculations and runs the ConvinceSense engine. It sends back a constant stream of metric, emotion, convincingness and transcript messages that keep the dashboard alive. You ask for a report and the backend gets all from the analysis and calls a local language model to write the narrative. We render to pdf and json. No unprocessed audio ever hits the disk, only the processed signals.

4.2 Architecture

The system is constructed in four layers: the browser, the FastAPI backend, two external model services, Deepgram for transcription and Ollama for the narrative, and a storage layer of SQLite plus the file system. The browser communicates with the backend in two ways: over REST to control analyses and reports, and over a full-duplex WebSocket for the live audio and the metric stream. Figure 1 shows the whole thing in sketch form.

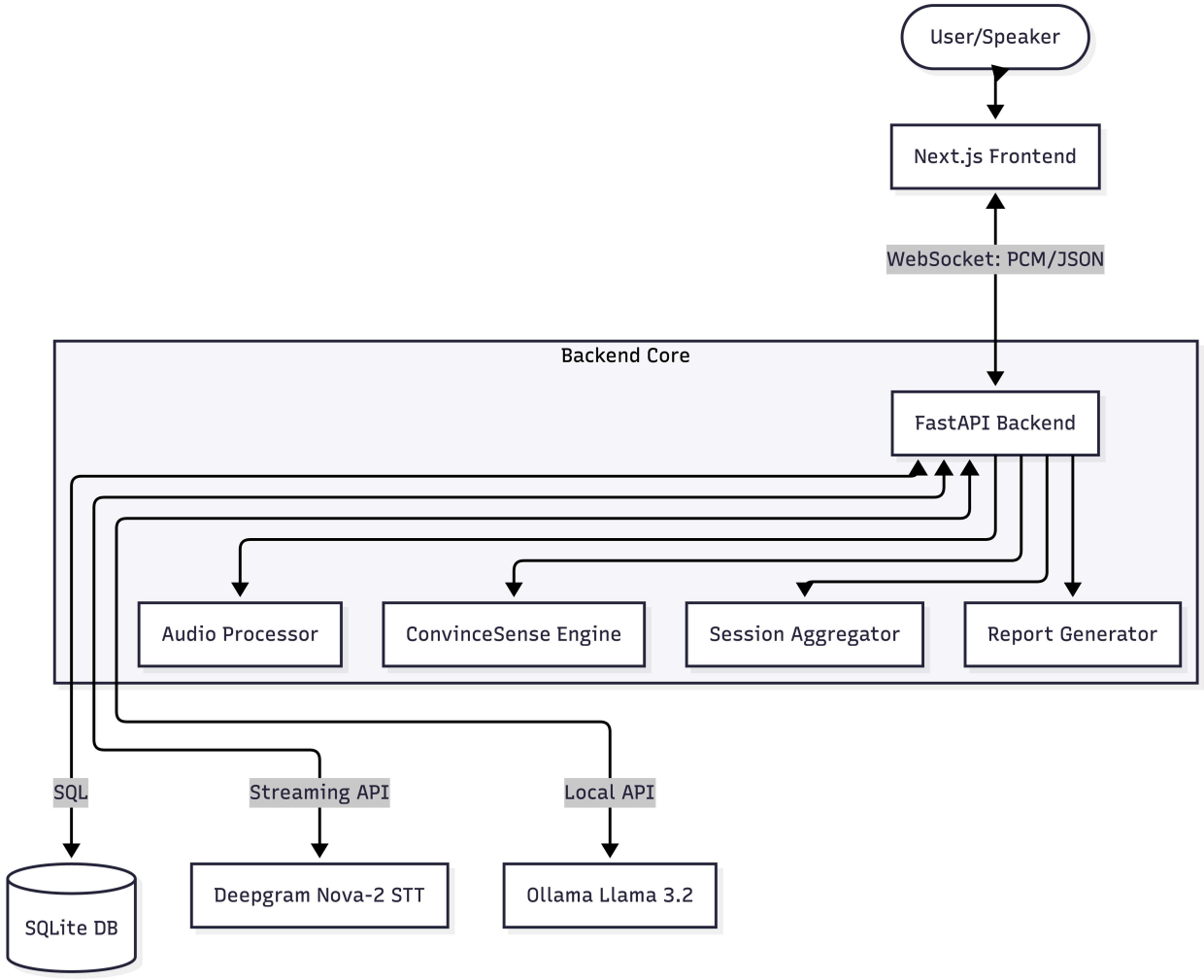

Figure 1: High-Level System Architecture

On the backend FastAPI exposes three REST routers, for analyses, sessions and reports, and the WebSocket endpoint for recording. The asynchronous design is not optional here: a single recording session has to receive binary frames, send them to the transcriber, run signal processing and fire off model inference, all at once, without ever stalling the event loop. We have set up Cross-Origin Resource Sharing so that the separate frontend can access the backend in development.

4.3 Core Concepts and Data Model

The system consists of three agents. Analysis is the overall entity the user creates. A named container e.g. "Interview Prep" containing one or more recording Sessions. A Session is a single continuous recording that captures all the metric frames, emotion events, filler events, and transcript segments during that period. A Report is generated on demand and gathers all sessions into an analysis; it is stored as files on disk and only its metadata and paths are stored in the

database. Figure 2 shows the hierarchy that allows one person to record three practice runs and get one report for all three.

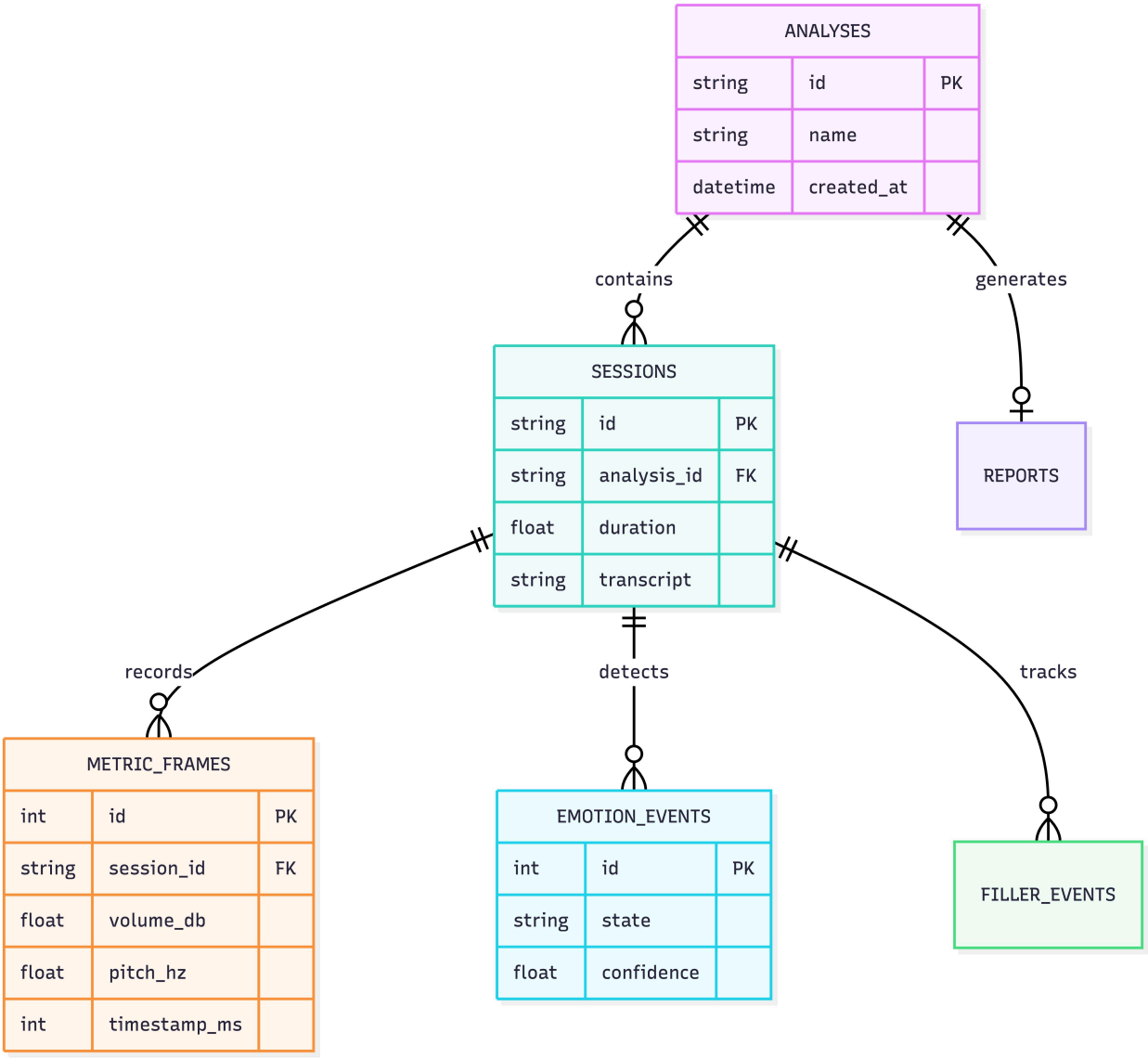

Figure 2: Entity Hierarchy and Data Model

One design decision worth mentioning here is that the system is intentionally single user and has no login. There is one user, so no accounts, no session tokens, no user IDs in the database. Every request is implicitly that one person's request. For a proof of concept this eliminates a whole layer of plumbing and keeps the work on the parts that are actually novel. On the other hand, if you add multi-user support later, you have to thread an owner through the schema and the API, which is exactly why we call it out as future work and not as a small change.

4.4 The Browser and the Live Dashboard

The frontend is a Next.js App Router app written in TypeScript and styled with Tailwind CSS and shadcn/ui components. It has an analysis manager to create and list analyses, an analysis detail page that lists sessions and contains the report controls, and a recording page where the dashboard lives. The dashboard is a collection of independent React components, an emotion badge, a volume meter, a pitch graph, an emotion-trend graph, a words-per-minute display, a filler counter and a scrolling transcript, arranged as shown in Figure 7. Each one subscribes to the messages it cares about through a custom hook, useSessionSocket, which owns the WebSocket and dispenses the latest metrics, emotion, transcript and timing state.

It was a good decision to keep all the connection logic inside that one hook. The dashboard components are still dumb and presentational, just rendering what the hook gives them, while the hook takes care of the dirty details: connecting when recording starts, buffering the latest values, expanding the running transcript, and tearing down the socket when recording stops. The words-per-minute number is calculated client-side from a rolling sixty second window of active speech, not the whole session, so it tracks the speaker's current speed and doesn't sag during a long pause. Doing that math on the client also reduces the amount of chatter the server has to send.

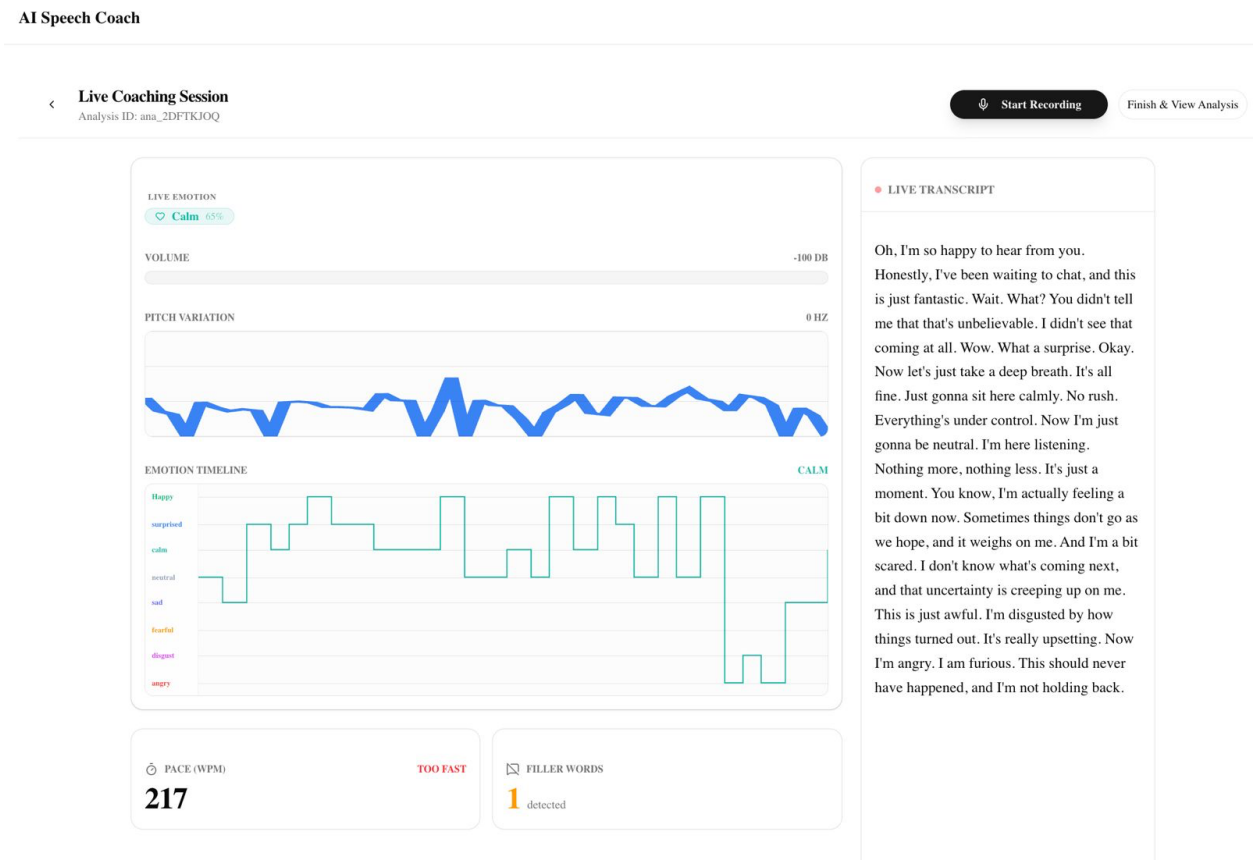

Figure 7: Live Coaching Dashboard Layout

4.5 Capturing and Streaming Audio

The Web Audio API on the client grabs the microphone when the user allows. The float32 samples are converted to 16-bit PCM and buffered into frames of about 250 milliseconds that are sent as binary WebSocket messages. At 44.1 kHz, a 250 millisecond frame is some eleven thousand samples, or some twenty two kilobytes. The standard scaling is the conversion from float to integer shown below. Again, no audio is written to disk in the browser, but frames are sent and then dropped from memory.

```
// Float32 [-1, 1]  ->  Int16 PCM
int16 = max(-32768, min(32767, Math.round(float32 * 32768)))
// ~250 ms frame @ 44100 Hz  ->  11025 samples  ->  22050 bytes
```

It is not a random number we picked, but a compromise we reached with the frame size of 250 milliseconds. Shorter frames give snappier metrics but a lot more messages and more per-frame overhead. Make them longer and the dashboard starts to feel laggy and the silence detection gets coarse. A quarter of a second is the sweet spot: fast enough that the meters feel live, big enough that librosa has real signal to work with on each frame, and convenient because eight of them make the two-second emotion window and twelve make the three-second scoring window with no awkward remainders.

4.6 The WebSocket Protocol

The socket is binary audio up, JSON down. When a connection arrives, the backend's session handler accepts it, writes a session row, spins up the audio processor, the ConvinceSense engine, the session aggregator and the Deepgram streamer, and drops into a receive loop. It calculates the metrics per frame, sends the frame to Deepgram, starts emotion and convincingness inference on a worker thread and pushes the resulting messages back. The heart of the loop is something like this:

```
while True:
    data = await websocket.receive_bytes()
    metrics = processor.process(data)                  # librosa DSP
    elapsed_ms = int((loop.time() - start_time) * 1000)
    aggregator.add_metrics(metrics, elapsed_ms)
    await deepgram.send(data)                           # streaming STT
    pcm = np.frombuffer(data, np.int16).astype(np.float32) / 32768.0
    asyncio.create_task(process_av(pcm, elapsed_ms))    # emotion + score
    await websocket.send_json({
        'type': 'metrics', 'pitch_hz': metrics.pitch_hz,
        'volume_db': metrics.volume_db, 'rms_energy': metrics.rms_energy,
        'is_silent': metrics.is_silent, 'timestamp_ms': elapsed_ms })
```

Table 6 presents the messages delivered by the backend. The trick that keeps everything smooth is firing the heavy inference as a detached asyncio task that wraps a worker thread. This way,

emotion and convincingness results come in when they're ready, but never hold up the quarter-second metric stream.

Message type	Payload	Cadence
metrics	pitch_hz, volume_db, rms_energy, is_silent	every ~250 ms
transcript	text, is_final, words, timestamp_ms	on Deepgram result
emotion	state, confidence, timestamp_ms	every ~2 s
score	value (1-5), confidence, timestamp_ms	every ~3 s
error	message	on recoverable error

Table 6: WebSocket Message Protocol (Backend to Browser)

4.7 Acoustic Feature Extraction

The 250ms frames are decoded to a float32 array and passed to AudioProcessor which computes the frame-level metrics using librosa. The root-mean-square energy is an estimate of the volume in decibels. The silence decision has a little hysteresis: a frame below the energy threshold has to repeat for two frames running before we call it silent, which stops the display from flickering on every short pause. The pitch is obtained with the YIN algorithm over the speech range of 80 to 400 Hz, the median is computed over the frame, and the last value is used when the frame is unvoiced. Zero crossing rate is used as a very rough proxy for noisiness and consonants. The pipeline is sketched in figure 4 and the processor core is shown below.

```
audio = np.frombuffer(chunk_bytes, np.int16).astype(np.float32) / 32768.0
rms = float(librosa.feature.rms(y=audio)[0, 0])
is_silent = (rms < SILENCE_THRESHOLD)  # confirmed over 2 frames
if not is_silent:
    volume_db = float(librosa.amplitude_to_db([rms])[0])
    pitches = librosa.yin(y=audio, fmin=80, fmax=400, sr=44100)
    pitch_hz = float(np.nanmedian(pitches))
zcr = float(librosa.feature.zero_crossing_rate(y=audio)[0, 0])
```

The fusion model wants something richer. So a separate AcousticExtractor works on a 3-second window resampled to 16 kHz. It extracts thirteen MFCC means and thirteen MFCC standard deviations, the mean and standard deviation of pitch from pYIN, the RMS energy and

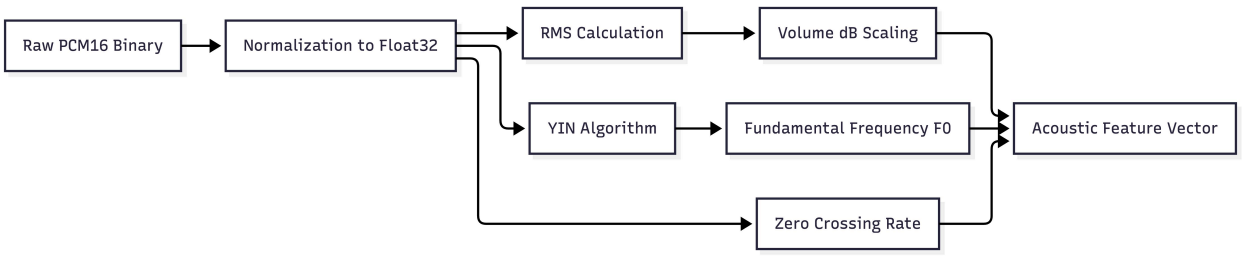

Figure 4: Acoustic Feature Extraction Pipeline

We use YIN for pitch, not the obvious FFT-peak approach, for a reason. Picking the highest frequency bin from a spectrum is being fooled all the time by harmonics and by noise and on a single human voice it tends to jump an octave at the worst times. YIN is autocorrelation-based

and is designed for precisely this monophonic case, and holds up much more stably across the 80-to-400-hertz band where speech resides. The price is that it leaves NaNs on unvoiced frames, so instead of allowing a gap to blank out the pitch graph, we take a median across the frame and carry the last good value forward.

4.8 Transcribing Speech

Deepgram Nova-3 does live transcription. We configured the connection to use 16-bit linear PCM at 44.1 kHz, mono, with smart formatting, interim results, and voice-activity events. Utterance-end timeout was set to 1 second. We enabled filler word recognition via an additional query parameter. If no Deepgram key is configured or the stream doesn't start, the pipeline will fallback to its own local Whisper 'small' model.

```
connection = client.listen.v1.connect(
    model='nova-3', encoding='linear16', sample_rate=44100,
    channels=1, smart_format=True, interim_results=True,
    utterance_end_ms=1000, vad_events=True,
    request_options={'additional_query_parameters':
                     {'filler_words': 'true'}})
```

The two types of result are handled differently. Interim results come fast, but can be updated. So they move the live transcript panel and make the speaker feel like the system is keeping up. The final results have word-level timestamps, and these are the ones we trust. They are appended to the session transcript, scanned for fillers, counted towards pace, and passed to the linguistic analyzer. Each final segment also is sent back to the client, so the transcript settles as the person speaks.

4.9 Measuring Pace and Filler Words

The speed is expressed in words per minute. That number on screen is pulled from that rolling sixty second window of active speech on the client. Dividing the words spoken in the window by the active time it covers and scaling to a minute keeps it honest about the current tempo. The aggregator also calculates an overall average at the end from the total word count and the duration on the server. Fillers are detected by lower-casing each word, stripping whatever is not a letter or a digit, and comparing it to a small set of usual suspects, "um", "uh", "like", "so", "right", "basically". Every hit is a timestamped event that feeds both the live counter and the after-the-fact breakdown.

```
fillers = {'um', 'uh', 'like', 'so', 'right', 'basically'}
for word in transcript_text.lower().split():
    clean = ''.join(c for c in word if c.isalnum())
    if clean in fillers:
        total_fillers += 1
        filler_events.append({'word': clean, 'timestamp_ms': ts})
```

4.10 Recognising Emotion in the Voice

The acoustic channel is based on a Wav2Vec2-XLSR model fine-tuned on RAVDESS, which classifies the voice into eight emotions: angry, calm, disgust, fearful, happy, neutral, sad and surprised. Inference runs on a two-second window, resampled to 16 kHz, which is the rate the model expects. Any window that is too quiet is skipped as silence. The model returns a probability distribution over the 8 classes and we take the highest label and confidence.

It is worth lingering where we were bitten, loading this checkpoint, because it is precisely research gap G4. The checkpoint advertises the standard architecture for sequence-classification, but its trained head is a custom one: a dense layer, a tanh, and an output layer on top of mean-pooled hidden states. But when you load it via some generic Hugging Face pipeline, that trained head is just silently discarded and replaced with random weights, so the predictions are garbage, they collapse onto a couple of classes regardless of what you say. The fix was to re-make the original head ourselves and load the trained weights directly into it, which we confirmed loads with zero missing or unexpected parameters.

```
class _Wav2Vec2ClassificationHead(nn.Module):
    def __init__(self, config):
        self.dense  = nn.Linear(config.hidden_size, config.hidden_size)
        self.dropout = nn.Dropout(config.final_dropout)
        self.output = nn.Linear(config.hidden_size, config.num_labels)
    def forward(self, x):
        x = self.dense(self.dropout(x)); x = torch.tanh(x)
        return self.output(self.dropout(x))
# pooled = wav2vec2(input_values)[0].mean(dim=1)  # mean pooling
# logits = classifier(pooled); probs = softmax(logits)
```

There are a couple of smaller details that are important as well. The raw label names of the model are normalised to a consistent lower-case set and the probabilities are folded into the shared emotion vocabulary so that the acoustic and linguistic sides speak the same language. The model requires 16 kHz so each 2-second window collected at the browser's 44.1 kHz is first resampled. And the dashboard keeps the last non-silent emotion between window boundaries so the badge changes at most every two seconds rather than flickering on every frame, which would just be distracting mid-sentence.

The eight RAVDESS classes do not fit well for coaching, it is fair to say. Instead, RAVDESS gives us angry, calm, disgust, fearful, happy, neutral, sad and surprised. A speaker would mostly want to know if they sounded nervous, flat, confident or excited. Some of these map across cleanly enough, calm to confident, fearful to nervous, but others, like disgust, rarely show up in a rehearsal and are not very actionable when they do. We retained the native labels rather than

a lossy remap, and treat the cleaner mapping to presentation-specific states as something to return to with a purpose-built model later.

4.11 Reading the Words

The linguistics side is working on the accumulated final transcript. DistilBERT fine-tuned on SST-2 outputs the sentiment positive or negative with a confidence. A weak negative is reread as neutral to compensate for the missing third class. The seven class DistilRoBERTa model gives the finer emotion as a full distribution over anger, disgust, fear, joy, neutral, sadness and surprise. Some rule-based detectors identify domain keywords, buying signals and hesitations, and categorize the intent into buckets such as pricing, comparison, objection, commitment and information. An intent confidence scales the density of trigger phrases by how long the utterance is. This is squashed into a compact numeric vector for the fusion model, like so:

```
# LinguisticFeatures.to_vector()
label_map = {'POSITIVE': 1, 'NEUTRAL': 0, 'NEGATIVE': -1}
return np.array([
    label_map[self.sentiment_label], self.sentiment_score,
    len(self.buying_signals), len(self.hesitations),
    len(self.detected_intents), self.intent_confidence],
    dtype=np.float32)
```

4.12 Fusing the Two Channels

ConvinceSense has two timers inside the single async loop. It buffers the incoming chunks and when enough have accumulated, it runs the voice-emotion model on a two-second window, and the convincingness scorer on a three-second window. That split is the entire point of gap G3. Two seconds is enough pitch contour and energy for tone, while a slightly longer window gathers enough words for the language side to have something to chew on. Below are the cadences and dispatch logic in Table 9.

```
self._emotion_buf.append(pcm_chunk); self._score_buf.append(pcm_chunk)
if len(self._emotion_buf) >= CHUNKS_PER_EMOTION:   # 8 chunks = 2 s
    emo_audio = np.concatenate(self._emotion_buf); self._emotion_buf = []
if len(self._score_buf) >= CHUNKS_PER_SCORE:       # 12 chunks = 3 s
    score_audio = np.concatenate(self._score_buf); self._score_buf = []
emo   = self._run_emotion(emo_audio, elapsed_s)    # Wav2Vec2-XLSR
score = self._run_score(score_audio, text, elapsed_s)  # fusion
```

Computation	Window	Roughly how often	Model
Live delivery metrics	250 ms	every 250 ms	librosa DSP
Voice emotion	2 s (8 chunks)	every ~2 s	Wav2Vec2-XLSR
Convincingness score	3 s (12 chunks)	every ~3 s	Random Forest
Transcription	streaming	interim + final	Deepgram Nova-3

Table 9: Multi-Rate Analysis Cadences

The acoustic and linguistic feature vectors are concatenated and fed into a scikit-learn Random Forest, which spits out an integer from one to five, mapped to Disengaged, Low Interest,

Neutral, Interested and Highly Interested. If there is no trained model on disk, then the engine silently reverts to a rule-based heuristic that raises or lowers a neutral baseline depending on sentiment, detected intents such as commitment or objection, and keyword counts. The prediction and the fallback are combined into a single method:

```
def predict(self, acoustic, linguistic):
    if self._clf is None:
        return self._heuristic(acoustic, linguistic)   # graceful fallback
    vec = concat(acoustic.to_vector(), linguistic.to_vector())
    label = self._clf.predict([vec])[0]
    conf  = max(self._clf.predict_proba([vec])[0])
    return int(self._le.inverse_transform([label])[0]), float(conf)
```

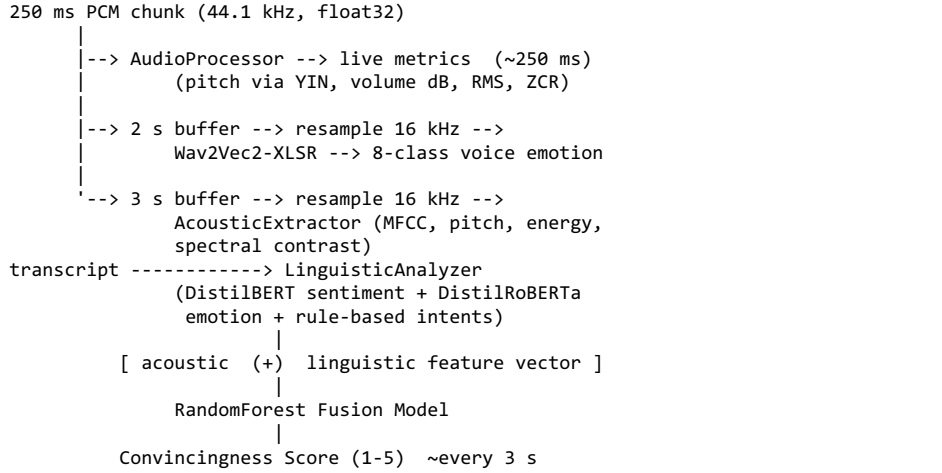

Figure 5: Multimodal Emotion and Convincingness Fusion Pipeline

We intentionally chose early fusion at the feature level instead of late fusion. The raw acoustic and linguistic vectors are concatenated so that the Random Forest learns how the two interact (e.g., positive words spoken with real vocal energy are more convincing than either signal alone). A forest is also well suited to this kind of short, heterogeneous vector, where the features are on wildly different scales, because trees do not care about monotonic scaling and handle mixed feature types well. Each score window is filled with whatever finalized Deepgram segments have been received since the previous score window, which is cleared after each score. This way, the successive numbers reflect recent speech rather than the whole talk.

4.13 Aggregating a Session

During a session a SessionAggregator quietly aggregates metric frames, filler events, emotion events, and per-window convincingness scores in memory. When the session is over it calculates the sums and writes all out. Average pace is the total words scaled to a minute over time. Volume and pitch are averaged only over voiced frames. The most frequent voice-emotion state is the prevailing emotion. The report's average convincingness, on a scale of one to five,

is translated into a clean headline number on a scale of zero to hundred, and that's the session confidence score. Figure 3 displays the entire data flow of a session.

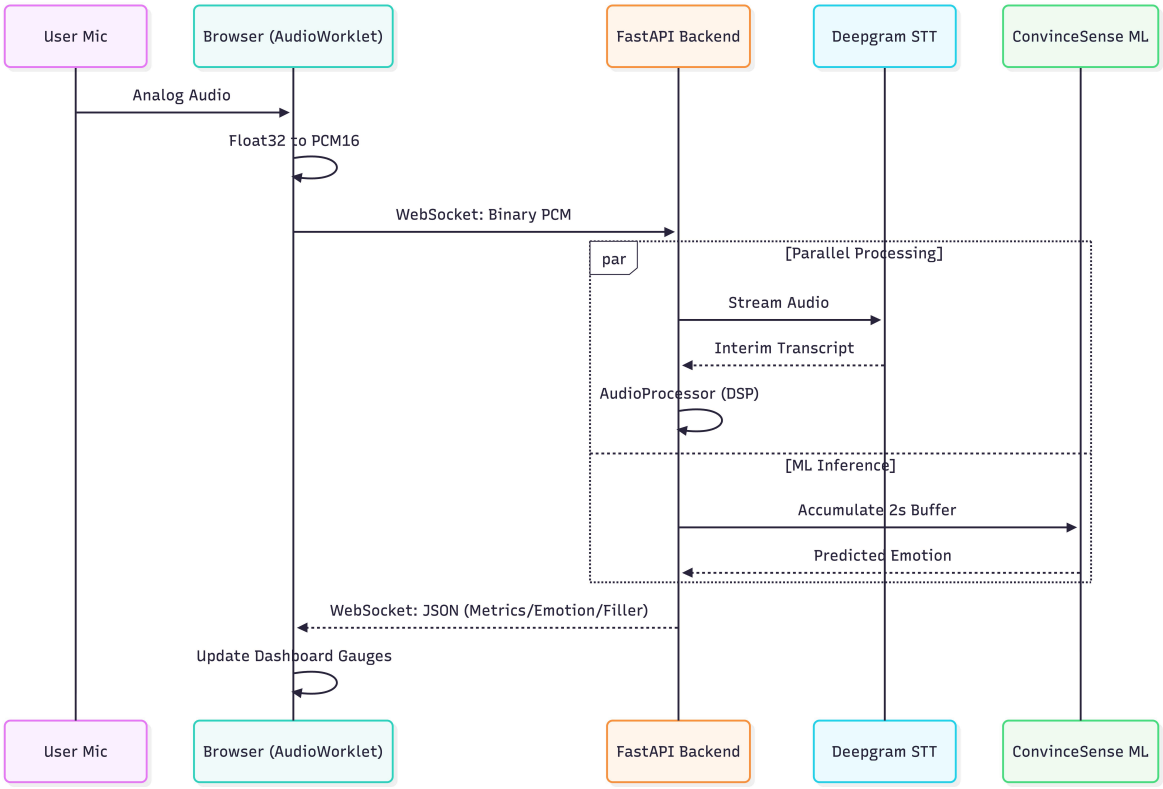

Figure 3: Real-Time Session Data Flow (Sequence)

4.14 Generating the Report

Report generation is on demand by design - and here's why. It only happens when the user wants it to, putting them in control of when they want feedback and preventing the machine from burning cycles unbidden. When it fires, the backend loads up all the sessions in the analysis and sums up the duration, the average pace, the number of fillers, the dominant emotion in the voice tone and a composite confidence score built from the convincingness values from each individual session, with a pace-and-filler fallback for any session that produced no scores. That composite is then re-mapped on to the one to five scale for display. Ollama sends a prompt with those metrics and a trimmed transcript to the local Llama 3.2 model, which produces a sectioned narrative, overall assessment, pace and delivery, fillers, voice tone and energy, and the top three improvements. We generate that into a PDF with ReportLab and a JSON file and store the report meta-data in the database. Figure 6 illustrates the flow. The prompt is constructed as follows:

```
user_prompt = f'''Generate a coaching report for "{ctx.analysis_name}".
Metrics:
 - Sessions: {ctx.session_count}; Duration: {ctx.total_duration_minutes:.1f} min
 - Convincingness: {ctx.convincingness_score}/5 ({ctx.convincingness_label})
```

```
   - Avg Pace: {ctx.avg_wpm} WPM;  Total Fillers: {ctx.total_fillers}
   - Dominant Voice-Tone Emotion: {ctx.dominant_emotion}
  Transcript:
   {ctx.full_transcript[:2000]}
  Structure: 1) OVERALL ASSESSMENT 2) PACE AND DELIVERY
   3) FILLER WORDS 4) VOICE TONE & ENERGY 5) TOP 3 IMPROVEMENTS'''

User clicks 'Generate Report'
   POST /api/analyses/{id}/report
Backend:
   1. SELECT sessions WHERE analysis_id = id
   2. Aggregate: duration, avg WPM, fillers, dominant
      emotion, confidence score (from convincingness)
   3. Map confidence (0-100) -> convincingness (1-5)
   4. Build prompt (metrics + transcript[:2000])
   5. POST Ollama /api/chat  (model = llama3.2)
   6. Write report.pdf (ReportLab) + report.json
   7. INSERT reports row (paths, model, score)
   8. Return download URL
```

Figure 6: Report Generation Flow

A small decision with a few payoffs: Make report generation an explicit button instead of something that fires automatically. It puts control of when you want to be judged in the hands of the speaker, which is more important than it sounds when you are practising something which makes you nervous. It avoids the need to spin up a slow local language model every time you take a short shot, which on a CPU would be a real drag. And it fits nicely with the multi-session idea: record a few tries, then request once a report that looks across them all.

When an analysis has multiple sessions, the report context joins them together rather than breaking them apart. The transcripts are joined together, fillers and durations are added up, the average pace is recalculated over the whole, and the per-session confidence scores are combined into one headline figure. This is what lets a user do several takes of the same talk and get a single report that reflects how they did overall, rather than a scattershot set of per-take summaries.

The PDF is meant to be read: it begins with the analysis name and headline numbers, then presents the narrative in its labelled sections, and ends with the ranked improvements. The JSON twin has the same story and the full raw metrics and the transcript, which is useful for debugging, for keeping, and for any future UI that might want to show the report interactively rather than as a flat document. Keeping the reports as files and only the paths and meta-data in the database keeps the database small and reports easy to open, back-up or pass along.

4.15 The Database

Structured data is stored in SQLite, and accessed via the async aiosqlite driver. Table 7 summarizes the schema, which consists of six tables. analyses has the top-level units, sessions

has per-session metadata and summaries, metric_frames, emotion_events and filler_events have the fine-grained time series, reports has report metadata and file paths. Foreign key indexes make the report-time aggregation queries fast. We never store raw audio only derived signals. This is how we satisfy privacy requirement NFR1. Here is a cropped view of the schema.

```
CREATE TABLE analyses (id TEXT PRIMARY KEY, name TEXT, status TEXT,
                       created_at TEXT, updated_at TEXT);
CREATE TABLE sessions (id TEXT PRIMARY KEY, analysis_id TEXT,
   started_at TEXT, ended_at TEXT, duration_seconds REAL,
   full_transcript TEXT, avg_wpm REAL, avg_volume_db REAL,
   avg_pitch_hz REAL, total_fillers INTEGER,
   dominant_emotion TEXT, confidence_score REAL, status TEXT);
CREATE TABLE metric_frames (id INTEGER PK, session_id TEXT,
   timestamp_ms INTEGER, wpm REAL, volume_db REAL, pitch_hz REAL,
   rms_energy REAL, zcr REAL, is_silent INTEGER);
CREATE TABLE emotion_events (..., state TEXT, confidence REAL, ...);
CREATE TABLE filler_events (..., word TEXT, timestamp_ms INTEGER);
CREATE TABLE reports (id TEXT PK, analysis_id TEXT, generated_at TEXT,
   confidence_score REAL, file_path_pdf TEXT, file_path_json TEXT,
   llm_model TEXT, ...);
```

Table	What it holds	Key columns
analyses	Top-level work units	id, name, status, created_at
sessions	Per-recording metadata + summary	id, analysis_id, avg_wpm, dominant_emotion, confidence_score
metric_frames	Per-250 ms time series	session_id, timestamp_ms, pitch_hz, volume_db, rms_energy, zcr
emotion_events	Voice-emotion changes	session_id, timestamp_ms, state, confidence
filler_events	Detected fillers	session_id, timestamp_ms, word
reports	Report metadata + file paths	id, analysis_id, file_path_pdf, llm_model, confidence_score

Table 7: SQLite Database Schema Summary

4.16 Technology Stack

Table 3 outlines the main technologies and their functions. The choices are towards an async Python backend for the concurrency, a modern React framework for the client, well-worn signal and machine-learning libraries and models we can host ourselves so the privacy and cost goals hold.

Layer	Technology	Role
Frontend	Next.js (App Router), TypeScript, Tailwind CSS, shadcn/ui	Web client and live dashboard
Audio capture	Web Audio API, WebSocket	PCM16 capture and streaming
Backend	FastAPI, Uvicorn	REST + WebSocket server
DSP	librosa, NumPy	Pitch, energy, MFCC, spectral features
Speech-to-text	Deepgram Nova-3 (Whisper fallback)	Streaming transcription
Voice emotion	Wav2Vec2-XLSR (RAVDESS, 8-class), PyTorch, Transformers	Acoustic emotion recognition
Linguistic	DistilBERT SST-2, DistilRoBERTa (7-class)	Sentiment and text emotion
Fusion	scikit-learn Random Forest, joblib	Convincingness score (1-5)
Report LLM	Ollama (Llama 3.2), httpx	Coaching narrative generation

Reporting	ReportLab, JSON	PDF and machine-readable report
Storage	SQLite via aiosqlite	Sessions and derived metrics

Table 3: Core Technology Stack of the System

Table 4 documents the emotion taxonomies of the two model channels, and Table 5 is the convincingness scale we employ throughout.

Channel	**Model**	**Emotion / Output Classes**
Acoustic (voice tone)	Wav2Vec2-XLSR (RAVDESS)	angry, calm, disgust, fearful, happy, neutral, sad, surprised
Linguistic (sentiment)	DistilBERT (SST-2)	positive, negative, neutral (synthesized)
Linguistic (text emotion)	DistilRoBERTa (GoEmotions)	anger, disgust, fear, joy, neutral, sadness, surprise

Table 4: Emotion Label Taxonomy of the Acoustic and Linguistic Models

Score	**Label**	**What it means**
1	Disengaged	Barely engaged / not persuasive
2	Low Interest	Weak engagement
3	Neutral	Baseline / neutral delivery
4	Interested	Engaged and fairly persuasive
5	Highly Interested	Highly engaged and persuasive

Table 5: Convincingness Score Scale and Interpretation

4.17 Handling Failure Gracefully

Since the system depends on a few heavy-weight externals, we designed the system to be resilient from the ground up, not as an afterthought. The WebSocket handler puts its receive loop in a try-except block, so if the client disconnects or a transient error occurs, the server does not go down; in either case a finally block ends the session and saves its data before closing the socket. The Deepgram connection is setup inside a try-except, so a failure to connect sends a clean error to the client and falls back instead of aborting. Each of the three model loaders catches and logs its own failures, returning nothing so that the engine can see that a model is missing and skip that channel rather than crashing. If a trained artifact is found, the fusion model loads it, otherwise it silently falls back to the heuristic. And the report generator catches Ollama errors and returns an explanatory placeholder instead of raising. So a missing or stopped language model degrades the report rather than the request. So that, all together, is how we meet NFR4.

```
try:
    while True:
        data = await websocket.receive_bytes()
        ...                                # process, transcribe, infer
except WebSocketDisconnect:
    logger.log('client disconnected')
except Exception as e:
    logger.error(f'session error: {e}')
finally:
    await deepgram.finish()
    await aggregator.save_to_db()        # always persist the session
    await websocket.close()
```

The common denominator here is that the system should bend before it breaks. None of the heavy parts, the transcriber, the emotion model, the fusion model, the language model, can be a single point of failure that takes the session down with it. If one is absent or misbehaves, the relevant channel dies and the rest continues, and session data still gets saved. That principle cost us a little extra code at every integration point, but it is the difference between a demo that works only when everything is perfect and one we were comfortable actually using.

4.18 Configuration and Deployment

We configure the system primarily through environment variables, which helps to keep secrets out of the code and to run the same code in different places. The Deepgram key, Ollama base URL and model, and database path are all taken from the environment with sensible defaults so it runs out of the box for local development. The main parameters are given in Table 11. In development, Next.js serves the frontend, and Uvicorn serves the backend with auto-reload. CORS is on, so the two can talk. The frontend is just a simple match for a static or edge host. The backend, with its model dependencies, wants a container or VM with enough memory. The local language model is an Ollama install that pulls the selected model onto a local port and serves it after a one-time pull.

Parameter	**What it controls**	**Default**
DEEPGRAM_API_KEY	Turns on Deepgram streaming transcription	(unset -> Whisper fallback)
OLLAMA_BASE_URL	Address of the local Ollama server	http://localhost:11434
OLLAMA_MODEL	Language model used for the report	llama3.2
DATABASE_URL	Path to the SQLite database file	backend/data/coach.db
REPORTS_DIR	Where generated reports are written	(created at startup)

Table 11: Principal Configuration Parameters

5. RESULTS AND FINDINGS

5.1 What We Built

We got a fully working prototype up and running end to end. A user can create a named analysis, start a session and watch a live dashboard showing: inferred voice emotion; current volume and pitch; emotion-trend graph; pace; running filler count and scrolling transcript. Stop, and the session is finalized and the derived data saved to SQLite. From there the user can request a coaching report, which is generated entirely on the local machine and downloaded as a PDF. The backend juggles transcription, signal processing and model inference for every frame without ever blocking the stream. This was the part we were most worried about going in.

5.2 Walking Through the App

A typical run goes like this: The user names an analysis from the home page which POSTs to the analyses endpoint and drops them on the detail page. They open the recording page and click Start Recording, the browser asks for the microphone, opens an audio context and opens the WebSocket. They talk. The dashboard is alive: volume meter and pitch graph four times a second; emotion badge every couple of seconds; convincingness signal every three or so; words-per-minute reading of the last minute of active speech; filler counter ticking up as fillers slip out. Hit Stop and the session saves, and the analysis page has a Generate Report button. Click it and the coaching PDF downloads once the local model is done.

The interface was developed on progressive disclosure principle. The dashboard provides only what is useful in the moment while the session is running, so the speaker is informed without being buried; the full depth waits for the report. The colour and movement of the meters are visual cues that tell you what's going on at a glance, and the connection status is clearly shown so you know the thing is live. That holding back in the session, with a full report afterwards, is to reflect the behaviour of a good coach: hardly ever interrupting you as you talk, and then sitting down and talking it over when you have finished.

5.3 How It Performs in Real Time

The system is on the multi-rate schedule of Table 9. Live metrics show up about every 250 milliseconds, the eight-class voice emotion about every two seconds and the convincingness score about every three. The heavy inference is pushed onto a worker thread. This keeps the metric stream smooth even while the emotion and fusion models are busy. To keep peak memory in check on a limited machine, the three models are loaded one after another at startup,

lightest first and the roughly 1.3-gig voice-emotion network last, which is what ultimately stopped the out-of-memory crashes we kept hitting when we loaded them all at once.

It's the split in cadence that makes it feel alive without feeling noisy. The volume meter and pitch graph provide that immediate this-is-listening feedback as you change your voice, riding the quarter-second stream, while the emotion badge and the convincingness signal move slowly enough to actually read and react to. In our own use this division seemed right: the fast signals reassure you the system is on, and the slow ones carry the judgments you would actually think about before pausing. Keeping the last emotion smooth within the window boundaries kills the flicker even more.

One small surprise was the importance of carry-forward of the emotion label to the product feeling. An early implementation would recompute and draw the badge as soon as any result returned, and the label would flicker between neighboring emotions often enough to be annoying mid-sentence. By keeping the last non-silent state between the two second boundaries, and only redrawing when a real new result arrived, the badge felt considered, rather than nervous, without actually slowing anything down underneath.

5.4 The Voice-Emotion Model

The rebuilt Wav2Vec2-XLSR head loaded its trained weights with no missing or unexpected parameters, and it produced sensible, spread-out probabilities across the eight classes. That's a world away from what we saw loading the same checkpoint with a generic pipeline, where the predictions just collapsed onto a couple of classes no matter the input. So the finding here is two-fold: gap G4 is real and easy to fall into, and the fix works. As would be expected from a model trained on acted studio speech, it is at its best with clean input, and gets jumpier when background noise and cheap microphones come into play.

5.5 Linguistic Analysis and Fusion

The linguistic analyzer reliably discriminates positive, neutral and negative and brings out useful intent and hesitation cues from the transcript. Folding the acoustic and linguistic features into one convincingness score gives a steadier, more readable signal than either channel alone, and the rule-based fallback means the system still returns something sensible when there is no trained fusion model or when a window is silent. The wording the user sees reflects that, since the Random Forest is trained on limited data, and we interpret the absolute score as a trend to watch, not a precise measurement.

5.6 The Generated Reports

Overall, the reports generated by Llama 3.2, run locally via Ollama, were coherent and promising, with metric-referenced sections as requested for overall assessment, pace and delivery, fillers, voice tone and energy, and the top three improvements. How long it takes to generate depends a lot on the hardware. On a CPU-only machine, with the analysis models also in memory, a report can take anything from tens of seconds to a couple of minutes. That's why the backend applies a generous timeout and limits the output length. The report is available as a readable pdf and a machine-readable json, which is handy for debugging and whatever the UI grows into next.

5.7 Functional Testing

We evaluated the system against a set of functional scenarios covering the main user journeys and key non-functional behaviour. Table 12 presents scenarios and our observations. Testing was performed manually against the live system, ensuring that the data flow was correct, the dashboard remained responsive, persistence actually occurred and the graceful degradation paths worked as intended.

ID	Scenario	Expected outcome	Result
T1	Create analysis and open recording page	Analysis stored; recording page loads	Pass
T2	Start recording and speak	Live metrics, emotion, transcript update	Pass
T3	Speak filler words	Filler counter increments correctly	Pass
T4	Stay silent	Silence detected; no spurious emotion	Pass
T5	Stop recording	Session finalized and saved to SQLite	Pass
T6	Generate report	PDF and JSON produced; metadata stored	Pass
T7	Run with no Deepgram key	Falls back without crashing	Pass
T8	Run with Ollama stopped	Explanatory placeholder returned	Pass

Table 12: Functional Test Scenarios and Outcomes

5.8 Meeting the Objectives

All the aims of Chapter 3 were fulfilled. Table 13 connects the objectives to the components and findings that deliver them, allowing us to easily follow the link between what we set out to do and what we built.

Objective	Delivered by	Status
O1 Low-latency capture and streaming	Web Audio API + WebSocket pipeline	Achieved
O2 Real-time delivery metrics	AudioProcessor + dashboard	Achieved
O3 Voice emotion from short windows	Wav2Vec2-XLSR with rebuilt head	Achieved
O4 Linguistic analysis and fusion	DistilBERT/DistilRoBERTa + Random Forest	Achieved
O5 Local AI coaching report	Ollama (Llama 3.2) + ReportLab	Achieved
O6 Privacy-preserving persistence	SQLite of derived signals; no raw audio	Achieved

Table 13: Achievement of Project Objectives

5.9 Privacy and Cost

One finding that we care about is that the privacy and cost goals actually held up. Only derived signals, the transcript, the metric frames, the emotion events get saved, no raw audio is ever written on disk, neither on the client nor on the server. Deepgram is the only external dependency and it even has a local Whisper fallback. The analysis and report stack runs locally. There's no fee per use, and the user's voice never leaves their machine.

It's worth saying what privacy-preserving doesn't mean here, so as not to oversell it: This is not anonymity because the transcript is still there. A transcript can be sensitive. What we guarantee is narrower and, we think, the part that actually worries people: the recording of your voice, the biometric itself, is never written anywhere. It only exists as a stream of frames passing through memory and once a frame has been processed it is gone. Usually, that's the reassurance someone practicing in a shared house or an office wanted.

5.10 What Still Falls Short

There's still a lot to fix, and it leads right into the next chapter. The Random Forest is trained on limited data so the convincingness number is indicative, not gospel. Voice-emotion recognition is sensitive to noise and microphone quality. The eight-class RAVDESS taxonomy, drawn from acted speech, does not map cleanly onto the states a speaker actually cares about (e.g., nervous, confident). Local language-model generation is slow on CPU-only devices. If the cloud service is involved, transcription quality depends on the network and the accent. And the whole thing is still single-user, audio only, unauthenticated, which is exactly what a proof of concept should be, and exactly what we would change next.

6. CONCLUSION

6.1 Conclusion

We wanted to build something that was accessible and private, that could provide speakers with real emotion-aware feedback while they are speaking and a solid coaching report afterwards. That's what the AI Speech Coach does. It captures audio from the browser and streams it to an async FastAPI backend that computes live metrics with librosa, reads voice emotion with a Wav2Vec2-XLSR model, reads transcript with DistilBERT and DistilRoBERTa, and fuses both into one convincingness score with a Random Forest, all running on a multi-rate schedule. The collected data is then converted to a warm, specific report using a local Llama 3.2 model in PDF and JSON. The trick is that the whole analysis and report stack runs locally and just keeps derived signals, showing that rich, multimodal speech coaching doesn't need to mean paid APIs or holding onto someone's voice. The project accomplishes its goals and bridges the gaps identified in Chapter 3, and in doing so, the efforts to load SER checkpoints properly and maintain bounded model memory provided us with lessons that we expect will be applicable to other real-time AI systems.

6.2 Future Work

A few directions would extend this:

- Train the fusion model on a larger, purpose-built, and properly labelled dataset, and then test it against human judgments, so the convincingness score is better calibrated.
- Add optional video and body-language analysis, posture, gesture, eye-contact, for a fuller read on delivery.
- Extend the support for more languages with cross-lingual XLSR backbone, multilingual transcription and text models.
- Include cross-session progress tracking, personal pace and filler targets, trend charts and milestones so users can see themselves improving over time.
- Go beyond a single-user proof of concept to multi-user accounts and cloud sync while keeping the local-first privacy model.
- Make a native or progressive-web mobile client, so people can practice on any device.
- Speed up on-device language-model inference (quantized models, with optional GPU acceleration) to cut down on wait on the report.
- Let users correct the transcript and flag wrong emotion calls, feedback that could be used later to tune the models to the individual speaker.

6.3 Closing Thoughts

If this project proves anything, it proves that today you can build a truly useful, privacy-respecting AI speech coach, using open models and open libraries on ordinary hardware. Combine real-time signal processing, deep acoustic emotion recognition, transformer-based text analysis, multimodal fusion and a locally hosted language model into one coherent system and you have a practical way of making good communication feedback available to anyone, no matter their budget or connectivity. The things we learned, especially around loading models correctly, running analysis on more than one clock and keeping memory in check, apply well beyond this one app, and the architecture leaves a solid base for the work ahead.

WORK PLAN AND TIMELINE

Throughout the semester, we moved through the stages of the project, from the groundwork to the multimodal engine to the report and integration testing. Each stage built on the last and testing was done all along, not saved till the very end. The first phase locked down the requirements and the architecture; the middle phases built the streaming pipeline, the signal processing and the three model channels; and the final phases wired up report generation, ran the evaluation, and produced this document. The plan is detailed in Table 10.

Phase	Activity	Duration
1	Requirement analysis, architecture design, and stack selection	Weeks 1-2
2	Frontend scaffolding, microphone capture, and WebSocket streaming	Weeks 3-4
3	Backend DSP pipeline and Deepgram streaming transcription	Weeks 5-6
4	Voice-emotion model integration and classifier-head fix	Weeks 7-8
5	Linguistic analysis and Random-Forest fusion (ConvinceSense)	Weeks 9-10
6	Session persistence, aggregation, and dashboard integration	Weeks 11-12
7	LLM report generation, PDF/JSON export, and integration testing	Weeks 13-14
8	Evaluation, tuning, documentation, and report writing	Weeks 15-16

Table 10: Project Work Plan and Timeline

By doing it in stages, we could get the scary bits over with first. We had proven out the streaming and transcription pipeline before bolting on the heavier models, and we had discovered and fixed the voice-emotion checkpoint problem in its own phase rather than tripping over it in final integration.

REFERENCES
(References in IEEE format)

1. [1] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in Advances in Neural Information Processing Systems (NeurIPS), 2020.
2. [2] A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," in Proc. Interspeech, 2021.

[3] S. R. Livingstone and F. A. Russo, "The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS)," PLoS ONE, vol. 13, no. 5, e0196391, 2018.

[4] C. Busso et al., "IEMOCAP: Interactive emotional dyadic motion capture database," Language Resources and Evaluation, vol. 42, no. 4, pp. 335-359, 2008.

[5] Z. Ma, Z. Zheng, J. Ye et al., "emotion2vec: Self-supervised pre-training for speech emotion representation," in Findings of the Association for Computational Linguistics (ACL), 2024.

[6] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi, "GoEmotions: A dataset of fine-grained emotions," in Proc. 58th Annual Meeting of the ACL, 2020.

[7] J. Hartmann, "Emotion English DistilRoBERTa-base," 2022. [Online]. Available: https://huggingface.co/j-hartmann/emotion-english-distilroberta-base

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," in Proc. NeurIPS EMC^2 Workshop, 2019.

[9] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv:1907.11692, 2019.

[10] B. McFee et al., "librosa: Audio and music signal analysis in Python," in Proc. 14th Python in Science Conference (SciPy), 2015, pp. 18-25.

[11] A. de Cheveigne and H. Kawahara, "YIN, a fundamental frequency estimator for speech and music," Journal of the Acoustical Society of America, vol. 111, no. 4, pp. 1917-1930, 2002.

[12] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[13] A. Radford, J. W. Kim, T. Xu et al., "Robust speech recognition via large-scale weak supervision," in Proc. International Conference on Machine Learning (ICML), 2023.

[14] Meta AI, "Llama 3: Open foundation models," 2024. [Online]. Available: https://ai.meta.com/llama/

[15] S. Chen et al., "WavLM: Large-scale self-supervised pre-training for full stack speech processing," IEEE Journal of Selected Topics in Signal Processing, vol. 16, no. 6, 2022.

[16] W.-N. Hsu et al., "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," IEEE/ACM Trans. Audio, Speech, and Language Processing, vol. 29, 2021.

[17] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. EMNLP: System Demonstrations, 2020, pp. 38-45.

[18] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[19] Deepgram, "Nova-3 streaming speech-to-text," Deepgram Documentation, 2024. [Online]. Available: https://developers.deepgram.com/

[20] S. Ramirez, "FastAPI: Modern, fast web framework for building APIs with Python," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[21] Ollama, "Ollama: Run large language models locally," 2024. [Online]. Available: https://ollama.com/

[22] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.